

Red Social

Microservicios con Spring Boot 4 y Angular 21

Prueba tecnica · Fullstack intermedio Java / Angular
Repositorio · <https://github.com/AndresFelipeDiazG/social-network>
Autor · AndresFelipeDiazG · **Fecha** · 13 de septiembre de 2026

1. Que se entrega

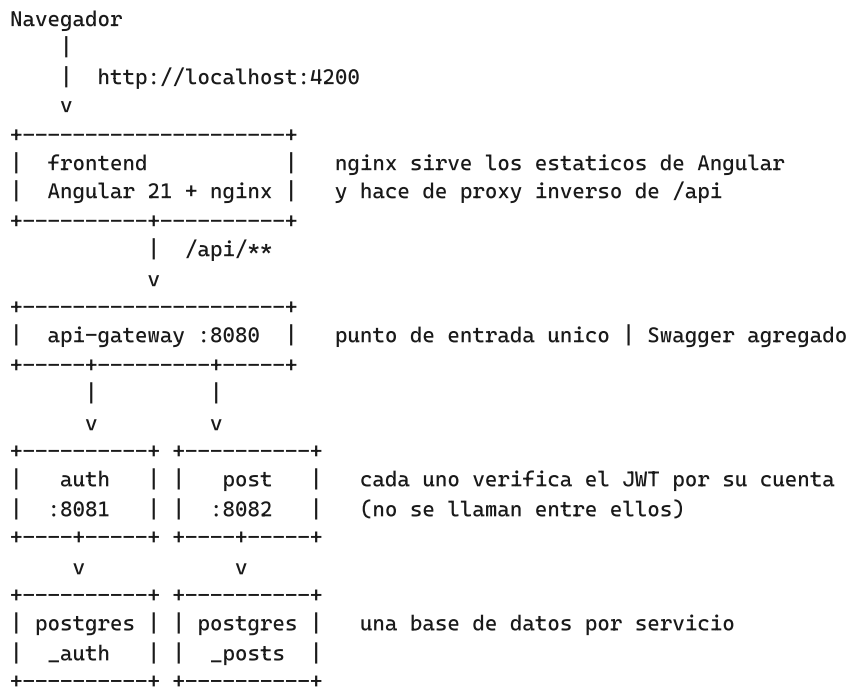
Los tres requisitos funcionales del enunciado, implementados de extremo a extremo:

Requisito	Donde vive	Estado
Autenticacion de usuario (usuario y clave)	<code>auth-service</code> emite el JWT · pantalla <code>/entrar</code>	Completo
Ver publicaciones (listar las de otros usuarios)	<code>post-service</code> , <code>GET /api/posts</code> · pantalla <code>/muro</code>	Completo
Crear una publicacion (mensaje, usuario y fecha)	<code>post-service</code> , <code>POST /api/posts</code> · compositor del muro	Completo

Y los requisitos tecnicos:

Pedido	Resuelto con
Backend en Java con Spring Boot	Spring Boot 4.1.1, tres servicios independientes
Login que entregue el JWT por <code>GET</code>	<code>GET /api/auth/login</code> con <code>Authorization: Basic</code>
Publicaciones: crear (<code>POST</code>) y listar (<code>GET</code>)	<code>post-service</code> , verificando el token localmente
Seeder de usuarios al arrancar, con una publicacion cada uno	Migraciones de Flyway <code>V2__seed_users.sql</code> y <code>V2__seed_posts.sql</code>
Dockerfile por servicio	Cuatro, multi-etapa, con usuario sin privilegios y <code>HEALTHCHECK</code>
PostgreSQL con ORM	Dos instancias de PostgreSQL 16 + Spring Data JPA (Hibernate 7)
Frontend Angular: login, listado y creacion	Angular 21, componentes autonomos y senales
Estado con Signals o NgRx SignalStore	Las dos cosas: <code>signalStore</code> de <code>@ngrx/signals</code> , construido sobre senales
<i>Deseable:</i> TypeScript, Swagger, pruebas unitarias, manejo de errores	TypeScript estricto · Swagger agregado en el gateway · 201 pruebas · <code>ProblemDetail</code> (RFC 7807)

2. Arquitectura



Flujo de la autenticacion

1. El navegador envia `GET /api/auth/login` con la cabecera `Authorization: Basic`.
2. `auth-service` busca el usuario, compara el hash BCrypt y firma un JWT HS256 con el secreto compartido.
3. El frontend guarda el token y lo adjunta como `Bearer` en cada peticion, mediante un interceptor.
4. `post-service` **verifica la firma con el mismo secreto, sin preguntar a nadie**. No hay llamada entre servicios ni sesion en memoria.

Por que la verificacion se repite en cada servicio. El gateway centraliza la entrada, pero si solo el gateway validara el token, cualquiera con acceso a la red interna podria llamar a `post-service` sin credenciales. Validar una firma es una operacion local de microsegundos: se paga barato y elimina la confianza implicita en la red.

Es comprobable: deteniendo `auth-service`, un token ya emitido sigue funcionando contra `post-service`.

3. Instalacion

Lo unico necesario es Docker. Java, Maven y Node solo hacen falta para desarrollar fuera de contenedores.

Requisito	Version	Comprobar
Docker	24 o superior	<code>docker --version</code>
Docker Compose	v2	<code>docker compose version</code>

Paso 1 · Clonar y crear el archivo de entorno

```
git clone https://github.com/AndresFelipeDiazG/social-network.git
cd social-network
cp .env.example .env
```

Paso 2 · Rellenar los tres secretos

El propio `.env` indica dentro los comandos para generarlos:

```
POSTGRES_AUTH_PASSWORD=...    # openssl rand -base64 24
POSTGRES_POSTS_PASSWORD=...   # openssl rand -base64 24
JWT_SECRET=...                # openssl rand -base64 48
```

El `.env` esta en `.gitignore` y nunca se sube. En `docker-compose.yml` los secretos se declaran como `${VAR:?mensaje}` en lugar de con un valor por defecto: si falta el archivo, Compose aborta diciendo exactamente que hacer. Con un valor por defecto, el fallo apareceria tres capas mas abajo como un error de autentificacion de PostgreSQL, que no explica nada.

Paso 3 · Levantar

```
docker compose up -d --build
```

Compose construye las cuatro imagenes y arranca los seis contenedores en orden: primero las dos bases de datos, y cada servicio espera a que la suya responda (`depends_on: condition: service_healthy`) antes de arrancar. Flyway crea las tablas y siembra los usuarios en el primer arranque. La primera vez tarda unos minutos por la descarga de dependencias; las siguientes son casi inmediatas gracias a la cache de capas de Docker.

```
docker compose ps      # los seis contenedores en healthy
docker compose logs -f  # seguir el arranque
```

4. Comprobar que funciona

Que	Donde
Aplicacion	http://localhost:4200
Swagger, API completa	http://localhost:8080/swagger-ui.html
Swagger de <code>auth-service</code>	http://localhost:8081/swagger-ui.html
Swagger de <code>post-service</code>	http://localhost:8082/swagger-ui.html

Usuarios predefinidos

Los crea Flyway al arrancar, con una publicacion cada uno. Comparten contrasena: `Password123!`

Usuario	Nombre	Identificador
<code>acorraa</code>	Ana Correa	<code>11111111-1111-1111-1111-111111111111</code>
<code>jmendoza</code>	Julian Mendoza	<code>22222222-2222-2222-2222-222222222222</code>
<code>lvargas</code>	Lucia Vargas	<code>33333333-3333-3333-3333-333333333333</code>
<code>dcastillo</code>	Diego Castillo	<code>44444444-4444-4444-4444-444444444444</code>

La pantalla de entrada muestra la cuenta de demostracion al pie: un clic en *Usar esta cuenta* rellena las dos casillas, y los otros tres usuarios estan a un clic al lado.

Desde la linea de comandos

```
./scripts/smoke-test.sh
```

Ocho comprobaciones de extremo a extremo contra el entorno levantado. O a mano:

```
# Login y token
curl -u acorraa:Password123! http://localhost:8080/api/auth/login

# Muro (sustituye TOKEN)
curl -H "Authorization: Bearer TOKEN" http://localhost:8080/api/posts

# Publicar
curl -X POST http://localhost:8080/api/posts \
  -H "Authorization: Bearer TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"message":"Hola mundo"}'
```

5. La API

Metodo	Ruta	Descripcion
GET	/api/auth/login	Login con <code>Authorization: Basic</code> . Devuelve el JWT. La que usa el frontend.
POST	/api/auth/login	Login con las credenciales en el cuerpo. Mismo resultado.
GET	/api/auth/me	Usuario correspondiente al token actual.
GET	/api/posts	Lista publicaciones. Parametros <code>scope</code> (<code>OTHERS</code> por defecto, <code>ALL</code> , <code>MINE</code>), <code>page</code> y <code>size</code> .
POST	/api/posts	Crea una publicacion. El autor sale del token; la fecha, del instante de guardado si no se envia.
PUT	/api/posts/{id}/like	Marca la publicacion con me gusta. Idempotente.
DELETE	/api/posts/{id}/like	Retira el me gusta. Tambien idempotente.
POST	/api/posts/images	Sube una imagen (multipart, maximo 2 MB) y devuelve su identificador.
GET	/api/posts/images/{id}	Devuelve los bytes de la imagen. Publica, ver la nota de abajo.

Hay dos variantes de login a proposito. El enunciado pide `GET` ; la convencion dice que una operacion con credenciales deberia ser `POST` . Se implementan las dos, con la misma logica detras. En ambas, las credenciales viajan en la cabecera y nunca en la URL, asi que no quedan registradas en los logs del servidor, en el historial del navegador ni en la cabecera `Referer` .

Los me gusta son idempotentes por disenio del esquema. La tabla `post_likes` tiene clave primaria (`post_id`, `user_id`) y la insercion usa `ON CONFLICT DO NOTHING` : marcar dos veces deja el mismo estado que marcar una. Comprobar antes de insertar dejaria una ventana entre la lectura y la escritura en la que dos peticiones simultaneas chocarian.

La descarga de imagenes es publica a proposito. El navegador pide las imagenes con una etiqueta `img` , que no envia la cabecera `Authorization` . El identificador es un UUID aleatorio, de modo que la direccion misma hace de credencial y solo se sirve lo que ya esta publicado. La alternativa (descargar con `fetch` y construir un blob) complica el cliente sin ganar nada real aqui.

Errores

Todos los servicios responden en formato `ProblemDetail` (RFC 7807), el estandar que Spring implementa de serie:

```
{ "type": "about:blank",
  "title": "Credenciales invalidas",
  "status": 401,
  "detail": "Usuario o contrasena incorrectos" }
```

Un usuario inexistente y una contraseña incorrecta devuelven **exactamente el mismo error**, para no confirmar que usuarios existen. Las excepciones no previstas se registran completas en el log y devuelven un mensaje generico: propagar el detalle expondria clases, consultas y rutas internas.

6. El frontend

Pantalla	Ruta	Contenido
Entrada	<code>/entrar</code>	Usuario y contraseña, con boton para revelarla, y la cuenta de demostracion al pie.
Muro	<code>/muro</code>	Publicaciones, compositor, filtros, buscador y carga incremental.

Angular 21 con componentes autonomos, senales y deteccion de cambios sin Zone.js. Las dos pantallas se cargan en diferido: el arranque baja 66 kB comprimidos mas la pantalla que toque, no las dos.

Estado

Dos `signalStore` de `@ngrx/signals`, con alcance distinto a proposito:

- **AuthStore**, en la raiz. Token, usuario y sesion persistida en `localStorage` junto a su instante de caducidad. Una sesion vencida se descarta al arrancar, sin esperar al primer 401: asi no se pinta el muro un instante antes de rebotar al login.
- **PostsStore**, provisto en la pagina del muro. Publicaciones, filtro, pagina y errores. Se destruye con el componente, de modo que al salir no queda el muro de la sesion anterior en memoria.

Decisiones de la interfaz

- **El filtro por defecto es «de otros»**. Es literalmente lo que pide el enunciado. *Todas* y *Mias* quedan como filtro explicito.
- **Paginado incremental, no paginas numeradas**. En un muro que se recorre hacia abajo, «ir a la pagina 3» no significa nada; el boton *Cargar mas publicaciones* con el contador *Mostrando X de Y* si. Descarta repetidas por `id`, porque una publicacion nueva desplaza la ventana entre dos peticiones.
- **El buscador filtra sobre lo ya cargado, y el texto lo dice**. El backend no expone busqueda; fingir que busca en todo el muro seria mentir en la interfaz.
- **Al publicar desde «de otros» se pasa a «todas»**. La propia publicacion no cabe en ese filtro y el usuario espera verla.
- **Tema claro, oscuro y del sistema**. Los tokens son propiedades personalizadas de CSS: cambiar de tema no recompila nada, y el modo «sistema» no escucha eventos, deja mandar a `prefers-color-scheme`.
- **Dos columnas en pantalla ancha**. El muro no llena una pantalla de 1440 px por si solo. La columna lateral se oculta por debajo de 1000 px y el muro se queda en 640, que es el ancho de linea comodo de leer.
- **Emojis en el compositor**. Paleta corta y fija, que inserta donde este el cursor y no al final. Un catalogo completo pide busqueda, categorias y varios cientos de kilobytes de datos para escribir un mensaje de 280 caracteres.

- **La barra superior solo muestra el avatar.** Cerrar sesion vive dentro del menu que se despliega al pulsarlo: la accion destructiva deja de estar a un clic de distancia por accidente.
- **Toda la interfaz usa Inter,** servida desde el propio bundle y no desde una CDN: la aplicacion tiene que verse igual sin salida a internet.
- **Los errores se muestran tal como llegan del backend.** El mensaje ya viene redactado alli, en un solo idioma y un solo sitio.

Configuracion en tiempo de ejecucion

Angular compila a JavaScript estatico y no puede leer variables de entorno. En lugar de recompilar la imagen por entorno, el contenedor genera `config.js` con `envsubst` antes de arrancar nginx, y la aplicacion lo lee al iniciar. **La misma imagen sirve para cualquier entorno:** cambiar el nombre de la aplicacion o el tamano de pagina es reiniciar el contenedor, no volver a construirlo.

7. Base de datos

Dos instancias de PostgreSQL 16, una por servicio. No comparten nada: ni esquemas cruzados, ni una base comun con dos usuarios.

Base de datos	Puerto	Tablas
<code>social_auth</code>	5433	<code>users</code>
<code>social_posts</code>	5434	<code>posts</code>

No hay clave ajena entre `posts` y `users`, y no es un descuido. Viven en bases de datos distintas y PostgreSQL no puede referenciar entre ellas. El vinculo son los UUID fijos, iguales en las dos siembras. Es la consecuencia visible —y el coste real— de tener una base de datos por servicio: se gana aislamiento y se pierde la integridad referencial declarativa, que pasa a ser responsabilidad de la aplicacion.

El nombre del autor se **copia** en cada publicacion en lugar de consultarse al leer. Una publicacion es un registro historico: el nombre que tenia su autor en ese momento es el dato correcto, igual que una factura guarda la direccion de entonces y no la actual. Y evita que listar el muro exija una llamada al otro servicio.

Scripts de la carpeta `db/`

Archivo	Contenido
<code>db/auth.sql</code>	Tabla <code>users</code> y los cuatro usuarios de prueba.
<code>db/posts.sql</code>	Tabla <code>posts</code> , sus indices y una publicacion por usuario.

```
psql -h localhost -p 5433 -U auth_service -d social_auth -f db/auth.sql
psql -h localhost -p 5434 -U post_service -d social_posts -f db/posts.sql
```

Estos dos archivos son una copia consciente. La fuente de verdad son las migraciones de Flyway que viven dentro de cada servicio, y son las que se ejecutan de verdad al arrancar. Los scripts de `db/` son su vista consolidada, porque el enunciado pide un script suelto como entregable. Se documenta en lugar de disimularse: en un proyecto real solo existirian las migraciones. Llevan `IF NOT EXISTS` y `ON CONFLICT DO NOTHING`, asi que se pueden ejecutar dos veces sin error.

8. Pruebas

Modulo	Pruebas	Que cubren
auth-service	38	Dominio, hash de contraseñas, emision del token, controlador y seguridad.
post-service	70	Validacion del mensaje, fechas, filtros del muro, paginacion y lectura del token.
api-gateway	19	Enrutado, propagacion de cabeceras y los metodos permitidos por CORS.
frontend	74	Los dos stores, formato de fechas, traduccion de errores y render de las dos pantallas.
Total	201	

```
# Backend, por servicio
cd backend/auth-service
./mvnw test      # unitarias y de slice: rapidas, sin Docker
./mvnw verify    # ademas las de integracion: levantan un PostgreSQL real

# Frontend
cd frontend
npm test         # Vitest, sin navegador
```

La separacion entre *Test y *IT es deliberada. Las unitarias corren con surefire y no necesitan Docker; las de integracion corren con failsafe y levantan PostgreSQL con Testcontainers. Mezclarlas obligaria a tener Docker para ejecutar cualquier prueba, que es exactamente como acaba desactivandose una suite entera.

9. Estructura del repositorio

```
social-network/
+-- docker-compose.yml      orquestacion de los seis contenedores
+-- .env.example            plantilla de configuracion
+-- README.md               documentacion tecnica
+-- backend/
|   +-- auth-service/       autenticacion y emision de JWT
|   +-- post-service/       publicaciones
|   +-- api-gateway/        punto de entrada unico y Swagger agregado
+-- frontend/              Angular 21 + NgRx SignalStore
+-- db/                    scripts SQL: esquema y usuarios de prueba
+-- scripts/               prueba de humo de extremo a extremo
+-- docs/                  este documento
```

Dentro de cada servicio del backend, la misma division:

```
src/main/java/com/social/<servicio>/
+-- domain/                sin una sola importacion de Spring
|   +-- model/              entidades y objetos de valor
|   +-- port/               interfaces que el dominio necesita
|   +-- usecase/            la logica de negocio
|   +-- exception/
+-- infrastructure/         todo lo que depende de un framework
|   +-- persistence/        JPA: entidades, repositorios, adaptadores
|   +-- security/           JWT, filtros, configuracion
|   +-- web/                controladores y DTO
|   +-- config/
```

La regla de dependencia va en una sola direccion: `infrastructure` conoce `domain`, nunca al contrario. Es comprobable de un vistazo, porque en `domain/` no hay ninguna anotacion de Spring. Eso es lo que permite probar la logica de negocio sin arrancar un contexto de aplicacion, y la razon de que la suite unitaria tarde segundos.

Monorepo, una rama por modulo

Los tres servicios y el frontend viven en un repositorio. Con cuatro modulos que se despliegan juntos y comparten el `docker-compose`, cuatro repositorios obligarian a coordinar cuatro *pull requests* para un cambio de contrato. El historial usa Conventional Commits, con una rama por modulo integrada mediante *pull request* y `--no-ff`, de modo que en el grafo se distingue donde empieza y donde acaba cada modulo.

10. Lo que se dejo fuera, y por que

- **Sin refresco de token.** El JWT dura una hora y al caducar la sesion se cierra avisando. Un *refresh token* exige almacenamiento, revocacion y rotacion: es la mitad de un servidor de identidad, y sobra aqui.
- **Sin registro de usuarios.** El enunciado pide autenticar, no crear cuentas. Los usuarios los siembra Flyway.
- **Las imagenes se guardan en la base de datos, no en un volumen.** Es una decision de entrega: asi el proyecto levanta con un solo comando y queda limpio con `docker compose down -v`, sin rutas montadas ni permisos que ajustar. Con volumen real de imagenes, lo correcto seria un almacen de objetos y guardar en la fila solo la referencia.

- **El token se guarda en `localStorage`** . Queda expuesto a XSS. La alternativa correcta es una cookie `HttpOnly` con `SameSite` , pero eso exige que el backend la emita, y el contrato pedido devuelve el token en el cuerpo de la respuesta. Es una limitacion conocida, no un descuido.
- **Una sola suite de integracion, no tambien e2e.** Con el tiempo disponible, las de integracion cubren mas riesgo por minuto invertido: verifican el contrato real contra PostgreSQL, mientras una suite e2e habria requerido levantar los seis contenedores y un navegador para comprobar lo mismo desde mas arriba.